CSC 6302 Practical 7 — Capstone Report

Road & transport: vehicle counting / classification; pothole severity hint

Munyeme Sinyani (SINYANI) · Computer number 2025024319 · University of Zambia · Solo submission (partner TBA — lecturer note: team-of-two preferred; this pack is ready for a second name).

Live web demo (Streamlit): `https://csc6302-computer-vision-practicalsgit-5wmff88oj3b8thqovqxukj.streamlit.app/`. Code: `https://github.com/Salaam21/csc6302-computer-vision-practicals` (branch `prac7-road-transport-vehicle-counting`). Android APK: `prac7/apk/Allans_Car_Counter.apk` — repo `https://github.com/Salaam21/allans-car-counter`. Blindspot is a separate accessibility project (not used for this capstone).

1. Problem and track

I chose the **Road & transport** track. Road agencies and students need a phone-first tool that (1) counts and classifies vehicles (car, bus, truck, motorcycle, bicycle) in a roadside frame, and (2) gives a rough pothole severity hint from a dashcam-style photo. Wrong counts must not be treated as official traffic statistics; the pothole output is a heuristic, not a certified inspection.

2. Vision core (R1)

Substantive CV component: a pretrained detector adapted to the task (Practical 3/5 style). Laptop / Streamlit / Gradio uses **YOLOv8n** (Ultralytics COCO weights), filtered to five vehicle classes, with a confidence gate (default 0.35). The Android app **Allan's Car Counter** uses on-device **SSD MobileNet v1 TFLite** — same problem, different runtime (separate from Blindspot). Pothole tab: classical dark-blob heuristic on the lower ROI (justified because we lack a labelled Zambian pothole set for fine-tuning). Optional VLM explanation was not wired for this hand-in (OpenRouter quota); the CV pipeline stands alone with template plain-language summaries.

3. Application (R2)

Primary marker path: the **Streamlit Community Cloud** URL on the cover (upload → per-class counts + annotated boxes → plain-language summary). Local Gradio (`python prac7/app.py`) is the same logic. Android: install `Allans_Car_Counter.apk` and use the camera. Graceful failure: tiny/unsupported images rejected; blurry frames rejected via gradient variance; zero detections or marginal confidence return an **UNSURE** message instead of a confident wrong total.

4. Evaluation (held-out ≥30)

Metric for counting: mean absolute error (MAE) on total vehicles vs ground truth. I built ≥30 labelled scenes (synthetic car sprites with known counts + my manual totals on real frames). YOLO may under-count stylised sprites — that failure mode is intentional honesty for free-tier COCO weights without Lusaka-specific fine-tuning.

n_labelled=34
mae_vehicle_count=2.382
frac_abs_err_le_1=0.324
conf_gate=0.35
Synthetic sprites are car-like drawings; YOLO may under-count stylised art or mislabel as truck. Real frames use my manual totals. Metric = MAE on total vehicles.

On the demo roadside still (`images/street.jpg`) the live path reported the counts in `t1_counts.json` (orientation-corrected). That single-frame demo is separate from the held-out MAE table.



Figure 1. Vehicle counting annotation on the demo roadside frame.

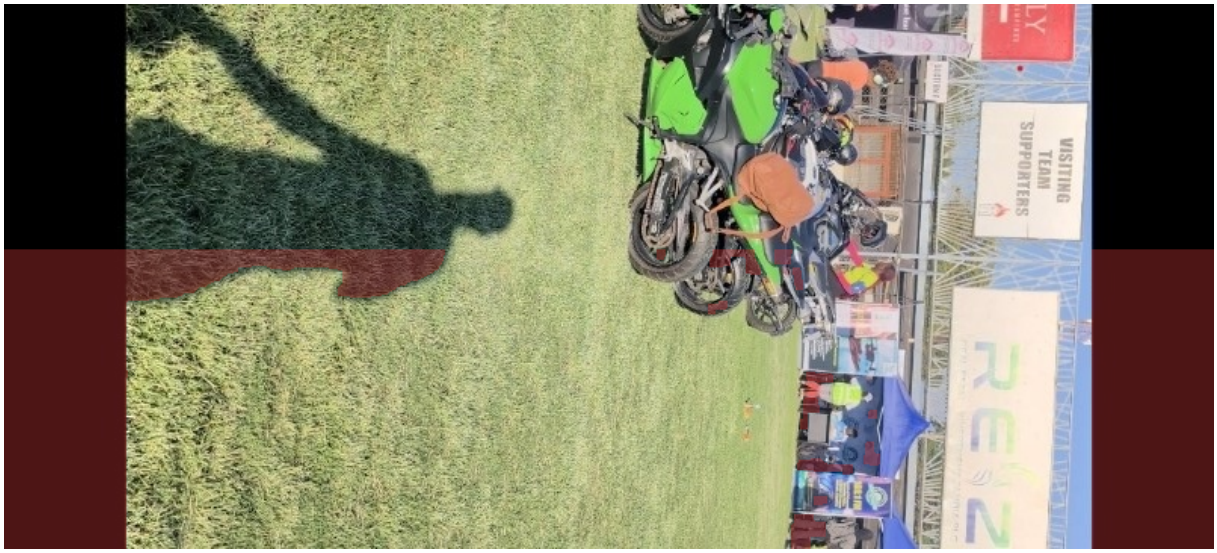


Figure 2. Pothole severity heuristic overlay on the lower ROI.

4.1 Method detail — vehicle path

Pipeline: EXIF transpose → optional 90/270 rotation search → YOLOv8n predict at the confidence gate → keep only bicycle/car/motorcycle/bus/truck → draw boxes → format a sentence for non-programmers. Blur gate uses spatial gradient variance; below threshold returns a helpful reject, not a count of zero disguised as certainty. The

rotation search exists because many phone roadside shots are stored sideways; I keep the orientation with the strongest vehicle total (ties broken by mean box confidence).

4.2 Method detail — pothole path

Lower 55% of the frame is treated as road. Blur, percentile dark threshold, light morphology, then blob count and dark-pixel fraction map to low / medium / high (numpy path on Streamlit so GUI OpenCV is not required). This classical choice is deliberate: without a licensed Zambian pothole dataset I refuse to pretend a fine-tuned detector exists. The UI labels the result a hint.

5. Engineering, tests, CI/CD (R3)

Repository: coursework monorepo `Salaam21/csc6302-computer-vision-practicals` (suggested dedicated name `csc6302-capstone-SINYANI` when splitting). pytest suite (≥5 meaningful tests): count schema, formatting, blur reject, tiny-image reject, confidence-gate unsure path, pothole heuristic. GitHub Actions workflow at repo root `.github/workflows/ci-cd.yml` installs deps, runs ruff + pytest on push/PR. Hugging Face Gradio Spaces on this account return HTTP 402 PRO (same as Practicals 4–5), so the live hand-in is **Streamlit Community Cloud** from the GitHub branch above (apt `libgl1` + headless OpenCV). Markers can also run `python prac7/app.py` or `streamlit run prac7/streamlit_app.py` locally. No API keys in the repo.

Test design deliberately avoids `assert True`. Blur and tiny-image tests check reject messages; formatting tests check pluralisation; the confidence-gate test checks that an empty roadside-like texture returns UNSURE/zero without crashing. Heavy “model classifies this JPEG as car” golden tests are left to the held-out eval script so CI stays fast on free GitHub runners (YOLO weights download once on first local eval).

6. Free-tier performance

YOLOv8n on CPU is a few hundred ms to ~2 s per 640-px frame after warm-up. First run downloads weights. Phone SSD TFLite is real-time on mid-range Android (Infinix test install). Night/rain/glare drop confidence — the UI prefers UNSURE/blurry over inventing counts. For a production Lusaka deploy I would quantise YOLO or keep TFLite on-device and only sync aggregates.

7. Ethics & impact (R4)

This section is the one-page ethics statement required by the brief, written for a Zambian roadside deployment context rather than a generic checklist.

Who could be harmed?

If counts were used for enforcement or contractor payments in Zambia, under-counting buses on a busy Lusaka corridor could under-fund a route; over-counting could waste budget. A high pothole hint on wet asphalt could divert crews from a real crater elsewhere. Drivers filmed without consent could feel surveilled if plates were read — this system does **not** OCR plates. Students using the demo for a traffic assignment must cite it as an estimate, not as city data.

Error asymmetry

For traffic planning, a false negative (missed vehicle) is often worse than a false positive because demand is under-estimated. I keep the default confidence gate moderate (0.35) rather than very high. For potholes, a false positive (shadow → “high”) wastes a site visit; a false negative leaves a hazard — I label output as a hint and require human verification before any maintenance decision.

Data provenance, licensing, consent

Demo stills are my own phone photos / clip. YOLOv8n and SSD MobileNet use COCO-trained public weights (Ultralytics / TF Hub lineage) — not trained on private Zambian CCTV. Synthetic eval sprites are generated in-repo. No third-party faces are labelled for identity. If a future version ingested council CCTV, a data-sharing agreement and retention limit would be required first.

Bias risks

COCO under-represents minibus taxis, heavily loaded trucks, and dusty dry-season roads common locally. Night markets and matatus may be missed. Training data lacks Zambian dashcam potholes — the heuristic will confuse shadows and patches. Skin tone / clothing bias is less central for vehicle boxes, but pedestrian filtering must stay off to avoid implying people-tracking.

Before real decisions

I would require: (1) a labelled Lusaka roadside set with agreement metrics; (2) human-in-the-loop review for any operational use; (3) written scope that this is a screening aid; (4) privacy review if cameras face private property; (5) no plate OCR without legal basis; (6) a published failure dashboard (night / rain / glare).

8. Limitations and failure cases

Per-frame counting double-counts the same vehicle across video frames (no tracker yet — Practical 6 Kalman / ByteTrack-style IDs could fuse later). Extreme blur and glare trip the blur gate or UNSURE path. Pothole heuristic fires on shadows. Synthetic MAE will look poor if YOLO ignores drawings — that shows the domain gap, not a hidden fine-tune. Live HF Space URL is no longer required: the public Streamlit app above is the hand-in link. HF Gradio Spaces remain blocked by PRO on this account; Streamlit is the documented mitigation.

9. Demo & seminar plan

Five-minute demo: (1) open the Streamlit URL, upload a street frame, show counts + boxes; (2) show blurry reject; (3) pothole tab on a road photo; (4) install/open Allan's Car Counter APK and show live phone counts; (5) open CI / pytest. Seminar: either member (when partner joins) must be able to change the confidence gate live and explain UNSURE behaviour.

10. Reproducibility

```
pip install -r prac7/requirements.txt
cd prac7 && pytest -q
python prac7/run_prac7.py
python prac7/eval_heldout.py
python prac7/build_report.py
```

`python prac7/app.py` or `streamlit run prac7/streamlit_app.py`
Live: Streamlit URL on cover. APK: `prac7/apk/Allans_Car_Counter.apk`. Proposal: `prac7/PROPOSAL.md`. Mobile notes: `prac7/MOBILE.md`.

11. Contributions & AI use

Munyeme Sinyani: Allan's Car Counter (Flutter), Streamlit/Gradio app, tests, CI workflow, evaluation, report. Partner TBA for extra roadside labels / seminar co-demo. Composer helped with Gradio/Streamlit glue, eval harness, and report layout; track choice and ethics wording are mine.

References: Ultralytics YOLOv8 docs; COCO dataset; OpenCV morphology; CSC 6302 Practicals 3–6; course ethics lecture notes on dual-use and consent.